



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I - III Semester(2025-26)
Faculty Name	Ms. Alluri Swathi
Student Name	Sivarapu Rishil
Roll Number	2520090174
Branch	CSIT
Course Outcome	4

Case Study Topic: Intelligent Smart City Infrastructure Optimization Platform Using Radix Sort

Work Done Summary:

Designed and implemented **Radix Sort** to efficiently sort a large dataset of emergency response vehicle IDs and city sensor data. In a smart city, thousands of emergency vehicles, traffic sensors, and surveillance cameras generate unique integer identifiers that need to be processed quickly for real-time dispatch, priority queuing, and data aggregation. Radix Sort handles these integers using a **non-comparison-based** approach – it sorts them digit by digit, from the least significant digit to the most significant digit, using counting sort as a stable subroutine. This provides **linear-time** sorting ($O(d \cdot n)$) which is far faster than comparison-based sorts ($O(n \log n)$) for fixed-width integer keys, making it ideal for high-throughput urban data pipelines.

Implementation Details :

- **Input Data:** An integer array representing emergency vehicle IDs, sensor codes, or zone identifiers.
- **Radix Sort Driver:** The `radixSort()` function determines the maximum number and iterates over digit positions (1, 10, 100, ...) using an `exp` variable.
- **Counting Sort Subroutine:** For each `exp`, `countingSort()` is called to sort the array based on the current digit:
 - Count the frequency of digits (0–9) in the current position.
 - Convert counts to cumulative positions to ensure stability.
 - Place elements into an output array in sorted order based on the digit.
 - Copy the output back into the original array.
- **Stability Maintenance:** Counting sort processes elements from right to left, which preserves the relative order of equal keys – crucial for correct multi-digit sorting.

- Main Method: Creates a sample list of 10 emergency fleet IDs, prints the original array, applies radix sort, and displays the sorted result.
- Output: The program clearly shows the unordered and ordered data, demonstrating the effectiveness of radix sort for city-scale integer datasets.

Result : Screenshots/Output:

```
=== UrbanAI - Smart City Data Sorting (Radix Sort) ===
Original Emergency Fleet Data: [1702, 945, 312, 1890, 542, 1101, 2300, 876, 65, 401]
Sorted Emergency Fleet Data (Ascending): [65, 312, 401, 542, 876, 945, 1101, 1702, 1890, 2300]
```

Time Complexity:

- *Counting Sort (per digit): $O(n + k)$, where n = number of elements and $k = 10$ (radix/base).
- *Radix Sort (overall): $O(d \cdot (n + k))$, where d = number of digits in the maximum number.
- *Since k is fixed at 10, this simplifies to $O(d \cdot n)$ for practical purposes.
- *Space Complexity: $O(n + k) = O(n)$ for the output and count arrays (extra space per digit pass).

GitHub Link : https://github.com/rishill10/DSA2_CO4

Student Signature	Faculty Signature

